SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3

7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3

18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3

30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I SANTHOSH R (192321007)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SANTHOSH R

1. Testing as an Engineering Activity

Introduction

Software testing is an important engineering activity used to improve software quality and detect defects systematically. Testing should not be performed randomly. It should be planned, designed, executed and evaluated using defined methods.

In the given scenario, the startup team uses random testing, which causes inconsistent results and missed defects. Therefore, testing should be redesigned as a structured engineering activity.

Problems with Random Testing

Random testing may result in:

- Important requirements being missed.
- Duplicate test cases.
- Poor test coverage.
- Inconsistent results.
- Difficulty in reproducing defects.
- Increased testing effort.
- Defects reaching production.

Structured Testing Approach

The testing process should begin with **requirement analysis**. Testers identify all functional and non-functional requirements and convert them into test conditions.

The next step is **test planning**, where the team defines the scope, resources, schedule, environment, risks and testing techniques.

After planning, testers perform **test case design**.

A test case should contain:

1. Test Case ID
2. Requirement ID
3. Test data
4. Preconditions
5. Test steps
6. Expected result

7. Actual result
8. Pass/Fail status

Testing Techniques

Suitable techniques include:

Equivalence Partitioning

Inputs are divided into valid and invalid groups.

Boundary Value Analysis

Values at and around boundaries are tested.

Decision Table Testing

Useful when several conditions determine the output.

State Transition Testing

Useful when system behavior changes according to its current state.

Example

Consider an online banking login system.

Tests should include:

- Valid username and password.
- Invalid username.
- Invalid password.
- Empty fields.
- Multiple failed attempts.
- Account lockout.

Instead of randomly trying different passwords, systematic test cases cover all important conditions.

Benefits

A structured approach:

- Improves defect detection.
- Increases test coverage.
- Reduces duplicated effort.
- Provides repeatable testing.
- Improves documentation.
- Makes regression testing easier.
- Improves overall software quality.

Conclusion

Testing should be treated as an engineering activity rather than random execution. Structured test design, planning and systematic testing techniques help identify defects effectively and improve the reliability of software.

---

2. Well-Defined Testing Process in a Banking System

Introduction

Banking systems are critical systems where software failures can cause financial loss. A well-defined testing process is therefore essential to ensure reliability, security and accuracy.

Testing Process

A standardized testing process consists of several stages.

1. Requirement Analysis

Testers study banking requirements such as:

- Account login.
- Balance checking.
- Fund transfer.
- Withdrawal.
- Transaction limits.
- Authentication.

Ambiguous requirements should be identified early.

2. Test Planning

A test plan defines:

- Scope.
- Resources.
- Testing schedule.
- Test environment.
- Risks.
- Entry and exit criteria.

3. Test Case Design

Test cases are designed from requirements.

For fund transfer, cases include:

- Valid transfer.
- Insufficient balance.
- Invalid account.
- Maximum transaction limit.
- Amount above limit.
- Network failure.

4. Test Execution

The designed test cases are executed and actual results are compared with expected results.

5. Defect Reporting

Failures are recorded with:

- Defect ID.
- Description.
- Steps to reproduce.
- Expected result.
- Actual result.
- Severity.
- Priority.

6. Retesting

After the developer fixes the defect, the tester executes the failed test case again.

7. Regression Testing

Existing functionality is tested to ensure that the new fix has not introduced another defect.

Example

For a ₹10,000 fund transfer:

Login → Select Beneficiary → Enter Amount → Authenticate → Transfer → Update Balance

Each stage is tested individually and as part of the complete transaction.

Benefits

A defined testing process:

- Reduces missed defects.
- Improves consistency.
- Provides traceability.
- Reduces production failures.
- Improves communication.
- Increases customer confidence.

Conclusion

A standardized testing process is essential for banking systems. Systematic testing throughout development helps identify defects early and ensures secure, reliable and accurate transactions.

---

3. Testing as a Structured Process in SDLC

Introduction

Software testing must be performed throughout the Software Development Life Cycle rather than only after coding. A structured process helps identify defects early and improves software quality.

Testing Across SDLC

Requirement Phase

Testers review requirements and identify:

- Missing requirements.
- Ambiguous statements.
- Incorrect assumptions.
- Unclear acceptance criteria.

Design Phase

Testers examine the architecture, interfaces, database design and data flow.

Coding Phase

Developers perform unit testing while testers prepare test cases and test data.

Integration Testing

Different modules are combined and tested.

For an e-commerce system:

Login → Product Search → Cart → Payment → Order

System Testing

The complete application is tested against its requirements.

The system is checked to determine whether it satisfies business requirements.

Example

Consider an e-commerce application.

A tester should verify:

- User registration.
- Login.
- Product search.
- Cart operations.
- Payment.
- Order confirmation.
- Order tracking.

Testing these functions systematically prevents defects from being discovered only after deployment.

Process Discipline

Process discipline means following:

- Standard test procedures.
- Standard test-case formats.
- Defined defect-reporting methods.
- Test entry and exit criteria.
- Regression procedures.

Benefits

Structured testing provides:

- Early defect detection.
- Better coverage.
- Consistent results.
- Easier defect tracking.
- Reduced testing cost.
- Improved software reliability.

Conclusion

Testing must be integrated into every SDLC phase. A disciplined process ensures that defects are detected early and that the final e-commerce system meets functional and quality requirements.

---

4. Error, Defect and Failure

Introduction

Error, defect and failure are important software-testing terms. Although they are related, each represents a different stage of a problem.

Error

An **error** is a human mistake made during requirements, design or coding.

Example:

A developer misunderstands a requirement and assumes that a password must contain exactly eight characters.

Defect

A **defect** is an incorrect implementation caused by an error.

Example:

The login program rejects all passwords longer than eight characters even though the requirement allows longer passwords.

Failure

A **failure** occurs when the software executes and produces an incorrect result.

Example:

A user enters a valid 12-character password but the system rejects it.

Relationship

The relationship can be represented as:

Human Error → Defect → Execution → Failure

Practical Example

Suppose the requirement says:

"Users can transfer up to ₹50,000 per day."

The programmer accidentally writes the limit as ₹5,000.

- Misunderstanding = Error.
- Incorrect limit in software = Defect.
- Customer being unable to transfer ₹10,000 = Failure.

Importance

Understanding these terms helps testers:

- Write clear defect reports.
- Identify the cause.
- Communicate with developers.
- Assign correct severity.
- Perform root-cause analysis.

Conclusion

An error is a human mistake, a defect is the incorrect software implementation caused by the error, and a failure is the incorrect behavior observed during execution. Understanding these concepts improves defect detection and resolution.

---

5. Early Defect Detection Principle

Introduction

Finding defects early is one of the important principles of software testing. The cost of fixing a defect generally increases when it is discovered at a later stage.

Early Testing

Testing activities should begin as early as possible in the SDLC.

Testers should participate during:

- Requirement analysis.
- Design review.
- Coding.
- Unit testing.
- Integration testing.
- System testing.

Example

Suppose a banking requirement states:

"Users can transfer money according to the daily transaction limit."

The requirement does not specify the exact limit.

If the tester identifies this ambiguity during requirement analysis, it can be corrected immediately.

If it is discovered after deployment, it may require:

- Requirement modification.
- Code modification.
- Retesting.
- Regression testing.
- Deployment.

Therefore, early detection reduces cost.

Techniques

Early defect detection can be improved through:

- Requirement reviews.
- Design reviews.
- Code reviews.
- Static analysis.
- Unit testing.
- Continuous integration.
- Automated testing.

Benefits

- Lower cost.
- Less rework.
- Faster development.
- Better requirements.
- Reduced production defects.
- Improved software quality.

Conclusion

Early defect detection is more effective and economical than finding defects after deployment. Involving testers early helps prevent defects instead of only detecting them later.

---

6. Tester Contribution During Requirement Analysis

Introduction

A tester should participate from the requirement phase instead of waiting until coding is completed. Early tester involvement helps identify ambiguity and missing requirements.

Role of Tester

During requirement analysis, testers:

- Review requirements.
- Identify ambiguous statements.
- Identify missing scenarios.
- Ask clarification questions.
- Define acceptance criteria.
- Identify possible risks.
- Prepare initial test conditions.

Healthcare Example

Suppose a healthcare application requirement states:

"Patient information should be updated quickly."

The word "quickly" is ambiguous.

The tester should ask:

- What is the maximum acceptable response time?
- How many users are expected?
- What happens if the update fails?
- Should changes be recorded in history?

These questions make the requirement testable.

Benefits

Early tester involvement:

- Prevents requirement defects.
- Improves requirement clarity.
- Reduces rework.
- Improves testability.
- Reduces development cost.
- Improves communication.

Example

For patient registration, the tester can identify requirements for:

- Patient name.
- Age.
- Gender.
- Contact information.
- Medical history.
- Mandatory fields.

- Invalid input handling.

Conclusion

Testers play an important role during requirement analysis. Their early involvement helps identify ambiguities before implementation and therefore improves software quality and reduces later defects.

---

7. Defects Originating from Ambiguous Requirements

Introduction

Software defects may originate at different stages of the software lifecycle. Ambiguous requirements are a major source because developers may interpret them differently.

Defect Origin

A defect may originate during:

1. Requirement analysis.
2. System design.
3. Coding.
4. Integration.
5. Testing.
6. Deployment.

Payroll Example

Suppose a payroll requirement states:

"Employees receive a bonus based on performance."

This is incomplete because it does not specify:

- Performance criteria.
- Bonus percentage.
- Minimum performance.
- Maximum bonus.
- Eligibility.

Different developers may implement different interpretations.

Result

This can produce:

Ambiguous Requirement → Incorrect Design → Incorrect Code → Wrong Salary → Production Failure

Preventive Measures

Requirements should be:

- Clear.
- Complete.
- Consistent.
- Testable.
- Unambiguous.
- Traceable.

Techniques

The team should conduct:

- Requirement reviews.
- Walkthroughs.
- Prototyping.
- Stakeholder discussions.
- Acceptance-criteria definition.

Conclusion

Many software defects originate before coding begins. Clear and testable requirements prevent misunderstandings and reduce the number of defects introduced during later SDLC stages.

---

8. Boundary Value Analysis

Introduction

Boundary Value Analysis is a black-box testing technique used to test values at and around the boundaries of input ranges. Many defects occur near boundaries because of incorrect comparison operators or off-by-one errors.

Example

Suppose a system accepts age from **18 to 60**.

Important test values are:

- 17
- 18

- 19
- 59
- 60
- 61

Expected Results

Input	Expected Result
17	Invalid
18	Valid
19	Valid
59	Valid
60	Valid
61	Invalid

Why Boundary Testing is Important

Normal values such as 30 or 40 may work correctly while boundary values may fail.

Possible defects include:

- Incorrect < instead of <=.
- Incorrect > instead of >=.
- Off-by-one errors.
- Overflow.
- Incorrect validation.

Practical Example

For an ATM that allows withdrawal between ₹100 and ₹20,000:

Test:

₹99, ₹100, ₹101, ₹19,999, ₹20,000, ₹20,001

This ensures the limits are handled correctly.

Benefits

Boundary testing:

- Detects input validation defects.
- Improves test coverage.
- Finds edge-case failures.
- Reduces production defects.

Conclusion

Boundary Value Analysis is highly effective for detecting defects near minimum and maximum limits. Testing values just below, at and just above boundaries improves software reliability.

---

9. Defect Repository for Better Test Case Design

Introduction

A defect repository stores information about defects found during previous testing and releases. Historical defect information can be used to improve future test cases.

Repository Information

A defect repository may contain:

- Defect ID.
- Module.
- Description.
- Severity.
- Priority.
- Root cause.
- Status.
- Resolution.
- Release information.

Using Historical Data

Suppose the repository shows that the payment module had many defects in previous releases.

Testers should increase testing for:

- Payment failures.
- Invalid cards.
- Timeout.
- Duplicate payment.
- Refund.
- Network interruption.

Defect Pattern Analysis

Testers can analyze:

- Defects by module.
- Defects by severity.

- Defects by release.
- Repeated defects.
- Root causes.

Example

If five previous releases contain login defects, the login module should receive stronger regression testing.

Benefits

The repository helps:

- Improve test coverage.
- Prioritize risky modules.
- Prevent repeated defects.
- Design regression tests.
- Reduce testing time.
- Improve software quality.

Conclusion

A defect repository should not merely store defects. Historical data should be analyzed and converted into improved test cases and regression tests to prevent recurring defects.

---

10. Functional Defect in ATM Withdrawal

Introduction

A functional defect occurs when a software function does not behave according to its specified requirement.

In an ATM system, incorrect withdrawal transaction logic is a functional defect.

Example

Suppose a customer has:

Account Balance = ₹20,000

The customer withdraws:

₹5,000

Expected balance:

₹15,000

If the system shows:

₹10,000

then the withdrawal logic is incorrect.

Possible Causes

- Incorrect balance calculation.
- Incorrect transaction logic.
- Incorrect database update.
- Duplicate deduction.
- Incorrect validation.
- Incorrect transaction amount.

Test Cases

The tester should test:

- Valid withdrawal.
- Insufficient balance.
- Maximum withdrawal.
- Minimum withdrawal.
- Invalid amount.
- ATM without sufficient cash.
- Network failure.
- Duplicate withdrawal.

Prevention

The defect could be prevented through:

- Requirement-based testing.
- Unit testing.
- Integration testing.
- Boundary testing.
- Decision table testing.
- Regression testing.

Example

For a withdrawal limit of ₹20,000:

Test:

₹19,999, ₹20,000 and ₹20,001

to verify the limit.

Conclusion

Incorrect ATM withdrawal processing is a functional defect because the system does not perform the required function correctly. Systematic functional, boundary and regression testing can prevent such defects.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand